Lecture 1

Machine Language (The Native Tongue)

The only language a computer truly understands is binary (1s and 0s).

* Form: Originally entered via physical switches (transistors) ; later evolved into hexadecimal for slightly better readability.

* The Catch: It is incredibly tedious, error-prone, and only feasible for tiny programs.

Assembly Language (Low-Level Abstraction)

To simplify things, assembly languages introduced mnemonics—short, human-readable codes that map directly to machine instructions.

* Examples: MOV (move data) or ADD (addition).

* Tool: Requires an assembler to translate the code back into binary.

* Limitations:

  * Low-Level: Still tied closely to the hardware.

  * Non-Portable: Code written for one processor won't work on another; it must be rewritten for different machines.

High-Level Languages (Abstraction & Portability)

Emerging in the 1950s/60s (e.g., Fortran, Algol), these languages revolutionized how we code by focusing on program flow rather than hardware mechanics.

*** Key Features:**

Introduced subroutines, loops, and conditionals.

* Hardware Independence: These languages are not tied to a specific CPU architecture.

* Portability: Ideally, a program written once can be "ported" and run on various different machines without a total rewrite.

Methods of Execution

High-level code must be translated into machine-specific instructions.

There are three primary ways to achieve this:

* Compilation: A Compiler translates the entire program into machine code before it ever runs.

This results in the fastest execution speed, though it requires a full re-translation every time you change the code.

* Interpretation: An Interpreter (like Python or MATLAB) reads and executes instructions "on-the-fly.

" While this is generally slower than compiled code, it is much better for rapid testing and development because it skips the compilation wait time.

* Virtual Machines: A "middle ground" approach (like Java) where code is compiled into an intermediate Byte-code. A Virtual Machine then interprets this byte-code, offering a smart balance of speed and portability.

High-Level Programming

The primary goal of a high-level language isn't just to make the computer work—it's to help humans build complex systems.

* Communication over Calculation: Programming is about communicating a software design to other programmers, not just the hardware.

* The Power of Modularity: High-level languages allow for "pluggable" modules. This means large, complex programs can be built from small, easy-to-understand pieces.

* Readability is King: A programmer’s focus should be on modularity and clarity.

You don't need to stress over every micro-second of speed; making the program run fast is primarily the compiler's job.

The C Programming Language

C is a general-purpose language, meaning it isn't limited to one field.

It is used in everything from operating systems to numerical computing and graphics. (washing machine, microwave, car systems).

* The "Middle" Language:

It provides a unique balance, offering high-level structures (loops, logic) while maintaining low-level power (direct memory and byte manipulation).

"Small" Language :

C is intentionally compact, containing only 32 keywords.

* Learning Curve: Because it is so small, a programmer can realistically master the entire language and use it regularly.

Spartan Design: It achieves this small size by excluding features found in other languages.

C has no built-in operations for complex objects (like lists), no automated memory management, and no built-in input/output (I/O) commands.

The Role of Functions and the Standard Library

Since the language itself is minimal, most of its power comes from functions stored in the Standard Library.

* Functionality via Libraries: Common tasks, such as printing to a screen, are handled by library functions like printf().

* Efficiency Tip: Instead of "reinventing the wheel" by writing your own code for basic tasks, you should use the standard library.

It provides implementations that are already tested, correct, and portable across different systems.

Examples of Famous C Standard Libraries:

<stdio.h> (Standard Input/Output):

Used for handling input and output operations, such as printing to the screen (printf) and reading user input (scanf).

<math.h> (Mathematics):

Used for complex mathematical operations, such as calculating square roots (sqrt), sine (sin), and cosine (cos).

<string.h> (String Handling):

Used for manipulating text strings, such as copying strings (strcpy) and measuring their length (strlen).

First C Program

This section breaks down the classic "Hello World" program to explain the essential building blocks of C.

The Core Components:

Functions and Variables

Every C program, regardless of complexity, is built from two things:

Functions: Groups of statements that perform specific tasks.

Variables: "Containers" that store data used during those tasks.

Breakdown of the "Hello World" Code

First C program: /*Hello World*/

```
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
}
```

Comments (/* ... */): Used for documentation.

They are ignored by the compiler.

Header Files (#include <stdio.h>):

This tells the compiler to look at the Standard Input/Output library.

Since C has no built-in I/O, you must include this to use functions like printf().

The main() Function:

This is the entry point of every C program. Execution starts here.

It returns an int (integer) to the operating system to signal if the program ran successfully (usually 0).

void indicates the function takes no arguments in this specific version.

Braces { }:

These define the "scope" or boundaries of the function.

printf() Statement:

A library function call that sends text to the screen.

Escape Characters: \n is used to start a new line.

Others include \t (tab)"\t: stands for Tab (a large indentation, typically equal to 4 spaces).

Semicolons (;):

These are mandatory. Because C is free-form (it ignores extra spaces or new lines), the semicolon is the only way the compiler knows a statement has ended.

Key Takeaway

Programming in C is about assembling library functions and your own logic into a structured flow.

The main() function is the "boss" that coordinates these actions.

A Numerical Example

```
#include <stdio.h>

int main() {
1 //    . Assign a fixed value
    float fahr = 100.0;
    float celsius;

2 //    . Calculate
    celsius = (5.0 / 9.0) * (fahr - 32.0);

3 //    . Print the result
    printf("Fahrenheit: %f\n", fahr);
    printf("Celsius: %f\n", celsius);

    return 0;
}
```

Why use float instead of int?

In programming, choosing the right "data type" is very important.

Here is the difference:

int (Integer): Used for whole numbers only (like 1, 5, -10).

It ignores anything after the decimal point.

The Problem: If you use int, the result of 5 / 9 becomes 0 because int cannot store 0.555.... This would make your whole equation wrong!

float (Floating Point): Used for numbers with decimals (like 3.14 or 37.7).

The Solution: Since temperature is rarely a perfect whole number (e.g., 98.6 or 37.2), float keeps the math accurate.

Simple Logic Breakdown:

Header (#include): Tells the computer how to use the printf function.

The Formula: We write 5.0 / 9.0 to force the computer to think in decimals.

If you write 5 / 9, the computer thinks you are using int and gives you 0.

The Output: %f is a placeholder that says "put a float number here."

This numerical example demonstrates how C handles data types, basic arithmetic.

1. Variable Declaration and Types

In C, all variables must be declared at the beginning of a block.

This program uses two fundamental types:

* int: For whole numbers (integers) like lower, upper, and step.

* float: For numbers with decimal points like fahr and celsius.

* Automatic Conversion: C can handle operations between different types (e.g., adding an int to a float) by automatically converting them.

The Art of Commenting

The goal of a comment is to provide insight that the code itself cannot convey.

Explain Intent: Use comments to clarify why something is being done and to point out subtle details within an algorithm.

Avoid Redundancy:

Do not use comments to simply restate what the code is already doing (avoid restating "code idioms").

Smart Naming: Choosing clear and descriptive identifiers (variable and function names) can significantly reduce the need for comments by making the code self-explanatory.

Readability: The ultimate focus is to produce readable code where the logic is apparent to other programmers.